Binary Search 5th June

✅ 1. Search Insert Position

Problem:

Given a sorted array `nums` and a `target`, return the index if the target is found. If not, return the index where it would be inserted in order.

Approach:

- Use classic binary search.
- If target is found → return index.
- Else → return `low`, where it would be inserted.

Fix in your code:

Your original code didn't handle the insert case correctly because it stopped early when `low < high`. The fix is to search until `low <= high` and return `low`.

Final Code:

```
int searchInsert(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return low;
}
```

✅ 2. Next Greatest Letter

Problem:

Given a list of sorted characters and a target, return the smallest character that is **greater than** target. Wrap around if needed.

Approach:

- Binary search for the **first letter > target**.
- Use `low % n` to handle wraparound.

Key Fix:

Use `low <= high` and adjust bounds carefully.

Code:

```
char nextGreatestLetter(vector<char>& letters, char target) {
    int low = 0, high = letters.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (letters[mid] <= target) low = mid + 1;
        else high = mid - 1;
    }
    return letters[low % letters.size()];
}
```

✅ 3. Is Perfect Square

Problem:

Determine whether a number is a perfect square.

Bug in your approach:

- You used a linear loop and faced overflow when `i*i` exceeds `INT_MAX`.

Fix:

- Use binary search and use `long long` to avoid overflow.

Optimal Code:

```
bool isPerfectSquare(int num) {
    long long low = 1, high = num;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        long long sq = mid * mid;
        if (sq == num) return true;
        else if (sq < num) low = mid + 1;
        else high = mid - 1;
    }
    return false;
}
```

✅ 4. Split Array into K Subarrays (General Discussion)

Problem:

Split array `nums` into `k` non-empty **contiguous** subarrays.

Variants Discussed:

- Count all ways (backtracking).
- Minimize the **maximum subarray sum** → Binary Search on answer space.

Approach (Binary Search on Value Range):

- Binary search the range between `max(nums)` and `sum(nums)`.
- Greedily check if it's possible to split into `k` parts with max sum ≤ mid.

✅ 5. Binary Search on Index/Value Range (Theory)

Discussed Concepts:

- Binary search is not always on array — often used on **value ranges**.
- Patterns:
  - **Min valid `x`** → shrink right: `high = mid - 1`
  - **Max valid `x`** → grow right: `low = mid + 1`

✅ 6. Kth Smallest Element in a Sorted Matrix

Problem:

Find the `k`th smallest element in a sorted matrix (rows & columns sorted).

Approach:

- Binary search the **value range**, not index.
- `low = matrix[0][0]`, `high = matrix[n-1][n-1]`.
- Count how many elements are ≤ mid using a bottom-left traversal.

Code:

```
int countLessEqual(vector<vector<int>>& matrix, int mid, int n) {
    int count = 0, row = n - 1, col = 0;
    while (row >= 0 && col < n) {
        if (matrix[row][col] <= mid) {
            count += row + 1;
            col++;
        } else {
            row--;
        }
    }
    return count;
}

int kthSmallest(vector<vector<int>>& matrix, int k) {
    int n = matrix.size();
    int low = matrix[0][0], high = matrix[n-1][n-1];
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (countLessEqual(matrix, mid, n) < k) low = mid + 1;
        else high = mid;
    }
```

```
    return low;
}
```

🔚 Summary Table

Problem	Binary Search On	Goal	Approach
Search Insert Position	Index range	Insert/find position	Classic BS
Next Greatest Letter	Index (mod wraparound)	Find next greater char	BS + % wrap
Is Perfect Square	Value range	Check if number is square	BS + `long long`
Split into K Subarrays	Value range (answers)	Minimize max sum	Greedy + BS
Kth Smallest in Matrix	Value range (matrix)	Find kth smallest number	Count + BS

🕵️ Clues That Suggest Binary Search

These phrases or characteristics often **indicate binary search is a good fit**:

Clue in the Problem	What It Means
“Find the first/last/smallest/largest…”	You're probably searching for a boundary (binary search on index or value)
“Sorted array / row / column”	Classic binary search — use index-based approach
“Minimize the maximum…” or “Maximize the minimum…”	Binary search on **answer space** (value range)
“Kth element” or “Top K”	Usually needs counting + binary search or heap
Monotonic behavior (function/array increases or decreases)	Perfect for binary search because of predictable transitions
Constraints too large for brute force (e.g., `10^5`, `10^9`)	Binary search helps reduce time to `O(log n)`

🔄 Common Scenarios & Patterns

✅ 1. Binary Search on Sorted Arrays

Classic usage — find element, first/last occurrence.

Examples:

- Search in Rotated Sorted Array
- Find First and Last Position of Target

cpp

```
CopyEdit
while (low <= high) {
    int mid = (low + high) / 2;
    if (nums[mid] == target) return mid;
    else if (nums[mid] < target) low = mid + 1;
    else high = mid - 1;
}
```

✅ 2. Binary Search on Answer Space (Value Range)

Used when the array isn't sorted but you're minimizing/maximizing something.

Examples:

- Split Array to Minimize Largest Sum
- Kth Smallest Element in Sorted Matrix
- Capacity to Ship Packages in D Days

```
cpp
CopyEdit
low = min_possible_answer;
high = max_possible_answer;

while (low < high) {
    int mid = (low + high) / 2;
    if (isValid(mid)) high = mid;
    else low = mid + 1;
}
```

✅ 3. Binary Search with Monotonic Functions

The function/condition changes from `false → true` or `true → false`.

Examples:

- Minimum Days to Make m Bouquets
- Find Minimum Speed to Arrive on Time

```
bool condition(int x); // returns true/false
```

✅ 4. Binary Search in 2D Matrices

Use either:

- Binary search across rows and columns (if matrix is fully sorted)
- Or count ≤ mid values using staircase pattern

Examples:

- Search 2D Matrix

- Kth Smallest in Sorted Matrix

✅ 5. Custom Binary Search Variants

You define your own condition on what is valid, and use binary search to find boundary.

Examples:

- Allocate Books
- Aggressive Cows (maximise minimum distance)

📘 Cheat Sheet: Binary Search Decision Tree

Question Says...	Likely Binary Search Type	Example
“Find an element”	Classic BS on index	`searchInsert()`
“Sorted matrix / array”	BS on index or values	`kthSmallest()`
“Minimize maximum…”	BS on answer space	`splitArray()`
“Find boundary (first/last true/false)”	Boundary BS	`firstBadVersion()`
“Too large for O(n)”	Try log(n) time	`isPerfectSquare()`
“Works when condition becomes true at some point”	Monotonic predicate	`minimumDays()`

🧠 Final Tips to Recognize Binary Search Problems:

1. **Check if the function/array is monotonic**:
    - If you can describe a condition that moves in one direction (true/false), it’s binary-searchable.
2. **Ask: can I model this problem as "Is x valid?"**
    - If yes → Binary search on `x`.
3. **Check constraints**:
    - If `n ≤ 10^5 or 10^9`, brute force is likely too slow → use binary search.